WORLD TRADE INSTITUTE · UNIVERSITY OF BERN

# Working with Text in Python

*A step-by-step reference: strings, files, your first text-analysis task, and a first look at regex*

Ahmad Alhineidi
Data Science Lab, University of Bern

[06.11.2026]

TODAY

# Six small steps

1

**Strings**

Store text and change it with built-in methods

2

**Loops**

Repeat an action for every item in a list

3

**Files**

Open, read and write .txt files

4

**The task**

Find the longest word in a text file

5

**Practice**

Short exercises, with expected output and answers

6

**Regex**

A first look at searching for patterns

BEFORE WE START

# How to read the code slides

CODE

```
print("Hello, WTI")    # this is a comment
```

OUTPUT

```
Hello, WTI
```

**Dark box**

The code. Type it into a Jupyter cell and run it with Shift + Enter.

**Light box**

What Python prints when you run that code.

**# comment**

Text after # is a note for humans. Python ignores it.

> Type the code yourself instead of copy-pasting: your fingers learn the syntax.

# 01

# Strings

*A string is text inside quotes*

PART 1 · STRINGS

# A variable stores a piece of text

CODE

```
text = "World Trade Organization"
print(text)
print(type(text))      # what kind of value?
print(len(text))       # count characters
print("The " + text)   # + glues strings
```

OUTPUT

```
World Trade Organization
<class 'str'>
24
The World Trade Organization
```

> Quotes make it text: "24" is a string, 24 is a number. len() counts spaces too.

PART 1 · STRINGS

# upper(), lower() and title()

CODE

```
text = "Agreement on Safeguards"
print(text.upper())
print(text.lower())
print(text.title())

# compare text, ignoring capitals
print("Trade" == "trade")
print("Trade".lower() == "trade")
```

OUTPUT

```
AGREEMENT ON SAFEGUARDS
agreement on safeguards
Agreement On Safeguards
False
True
```

> For Python, Trade and trade are different words. lower() first, then compare.

PART 1 · STRINGS

# strip() cleans the edges of a string

CODE

```
line = "   Article XX   "
print("[" + line + "]")
print("[" + line.strip() + "]")

line = "Article XX\n"     # \n = line break
print("[" + line.strip() + "]")
```

OUTPUT

```
[   Article XX   ]
[Article XX]
[Article XX]
```

> Every line you read from a file ends with \n. strip() removes it: you will use it a lot.

PART 1 · STRINGS

# count() and replace()

CODE

```
text = "trade in goods and trade in services"
print(text.count("trade"))
print(text.replace("trade", "commerce"))
print(text.count("Trade"))   # capitals matter!
```

OUTPUT

```
2
commerce in goods and commerce in services
0
```

> replace() gives a new string; text itself stays the same.

PART 1 · STRINGS

# Is it in there? in, startswith, find

CODE

```
title = "Article III: National Treatment"
print("National" in title)
print("national" in title)
print(title.startswith("Article"))
print(title.endswith("."))
print(title.find("III"))    # position
```

OUTPUT

```
True
False
True
False
8
```

> True / False answers are perfect inside an if. find() gives -1 if nothing is found.

PART 1 · STRINGS

# split() cuts text into a list of words

CODE

```
sentence = "Members shall accord national treatment"
words = sentence.split()    # cut at every space
print(words)
print(len(words))           # how many words?
print(words[0])             # the first word
```

OUTPUT

```
['Members', 'shall', 'accord', 'national', 'treatment']
5
Members
```

> split() is the heart of today's task: it turns a line into words.

PART 1 · STRINGS

# Cut at commas, glue back with join()

CODE

```
dates = "2024,2025,2026"
print(dates.split(","))    # cut at commas

words = ["trade", "in", "goods"]
print(" ".join(words))     # glue with spaces
print("-".join(words))     # glue with dashes
```

OUTPUT

```
['2024', '2025', '2026']
trade in goods
trade-in-goods
```

> join() is the opposite of split(): list in, one string out.

PART 1 · STRINGS

# Pick characters by position

CODE

```
word = "Agreement"
print(word[0])      # first character
print(word[-1])     # last character
print(word[0:5])    # positions 0 to 4
print(len(word))
```

OUTPUT

```
A
t
Agree
9
```

> Python counts from 0. In [0:5] the end position 5 is not included.

PART 1 · STRINGS

# Methods don't change the original

CODE

```
name = "wto"
name.upper()            # result is lost!
print(name)

name = name.upper()     # save the result
print(name)
```

OUTPUT

```
wto
WTO
```

> If nothing seems to happen, check that you stored the result with =.

PART 1 · STRINGS

# String methods at a glance

| CODE | PRINTS | CODE | PRINTS |
|---|---|---|---|
| `"wto".upper()` | `WTO` | `"tariff".find("f")` | `4` |
| `"WTO".lower()` | `wto` | `"GATT".startswith("G")` | `True` |
| `"trade law".title()` | `Trade Law` | `"a b c".split()` | `['a', 'b', 'c']` |
| `" hi ".strip()` | `hi` | `"-".join(["a", "b"])` | `a-b` |
| `"a-b".replace("-", " ")` | `a b` | `"1994".isdigit()` | `True` |
| `"tariff".count("f")` | `2` | `len("trade")` | `5` |

> isdigit() checks if all characters are digits. Also try isalpha() and isupper().

# 02

# Loops

*Do the same thing for every item*

PART 2 · LOOPS

# A for loop repeats for every item

CODE

```
words = ["trade", "tariff", "dispute"]

for word in words:
    print(word, len(word))

print("done")
```

OUTPUT

```
trade 5
tariff 6
dispute 7
done
```

> The colon : and the 4-space indent mark what repeats. done is not indented: it runs once.

PART 2 · LOOPS

# Add an if to choose which items

CODE

```
words = ["the", "Agreement", "on", "Safeguards"]

for word in words:
    if len(word) > 3:
        print(word)
```

OUTPUT

```
Agreement
Safeguards
```

> Each line ending with : adds 4 more spaces of indent to the block below it.

PART 2 · LOOPS

# A variable can remember across the loop

CODE

```
words = ["the", "Agreement", "on", "Safeguards"]

count = 0                  # 1. create before
for word in words:
    count = count + 1      # 2. update inside
print(count)               # 3. use after
```

OUTPUT

```
4
```

> Create, update, use. The longest-word task follows exactly this pattern.

# 03

# Files

*Open a text file, read it, write a new one*

PART 3 · FILES

# Our example file: trade.txt

TRADE.TXT · 3 LINES

```
The parties to this Agreement recognize that trade
should raise standards of living and ensure full employment.
Members shall treat like products (hereinafter, "products") equally.
```

### Where to save it

In the same folder as your notebook. Then the name trade.txt is enough.

### Make your own

Paste any text into a plain .txt file. Every example works on it.

PART 3 · FILES

# Open, read, close

CODE

```
f = open("trade.txt", "r")    # "r" = read
text = f.read()               # the whole file
f.close()                     # always close

print(text)
```

OUTPUT

```
The parties to this Agreement recognize that trade
should raise standards of living and ensure full employment.
Members shall treat like products (hereinafter, "products") equally.
```

> This works, but it is easy to forget close(). The next slide shows a safer way.

PART 3 · FILES

# with closes the file for you

CODE

```
with open("trade.txt", "r") as f:
    text = f.read()

# here the file is already closed
print(len(text))
print(text.count("Members"))
```

OUTPUT

```
181
1
```

> Always use with. Everything indented under it can use the file f.

PART 3 · FILES

# Read a file line by line

CODE

```
with open("trade.txt", "r") as f:
    for line in f:
        print(line)
```

OUTPUT

```
The parties to this Agreement recognize that trade

should raise standards of living and ensure full employment.

Members shall treat like products (hereinafter, "products") equally.
```

> Why the empty lines? Each line ends with \n, and print() adds another one.

PART 3 · FILES

# strip() removes the extra line breaks

CODE

```
with open("trade.txt", "r") as f:
    for line in f:
        line = line.strip()   # remove \n
        print(line)
```

OUTPUT

```
The parties to this Agreement recognize that trade
should raise standards of living and ensure full employment.
Members shall treat like products (hereinafter, "products") equally.
```

> Loop over f, strip() each line, then work with it.

PART 3 · FILES

# Three modes: read, write, append

| MODE | MEANING | FILE EXISTS | FILE MISSING |
|---|---|---|---|
| `"r"` | read | reads it | FileNotFoundError |
| `"w"` | write | **erases it**, starts empty | creates it |
| `"a"` | append | adds at the end | creates it |

**TEXT WITH É, Ü, Ç? ADD THE ENCODING**

```
with open("trade.txt", "r", encoding="utf-8") as f:
```

> If you leave the mode out, Python uses "r". Be careful with "w": the old content is gone.

PART 3 · FILES

# Write and append to a file

CODE

```
with open("notes.txt", "w") as f:
    f.write("First line\n")
    f.write("Second line\n")

with open("notes.txt", "a") as f:
    f.write("Third line\n")

with open("notes.txt", "r") as f:
    print(f.read())
```

OUTPUT

```
First line
Second line
Third line
```

write() does not add \n for you. Mode "w" erases old content.

PART 3 · FILES

# Error messages you will meet

| PYTHON SAYS | WHAT IT USUALLY MEANS |
| --- | --- |
| `FileNotFoundError: No such file or directory` | The file is not in the notebook's folder, or the name has a typo. |
| `IndentationError: expected an indented block` | The line after a colon needs 4 spaces of indent. |
| `SyntaxError: expected ':'` | You forgot the colon after for, if or with. |
| `NameError: name 'longest' is not defined` | You used a variable before creating it, or misspelled it. |
| `AttributeError: 'list' object has no attribute 'split'` | You used a string method on a list. Split a line, not a list. |

# 04

# The task

*Find the longest word in a text file*

PART 4 · THE TASK

# First, plan it in plain words

| IN PLAIN WORDS | IN PYTHON |
| --- | --- |
| 1. Start with an empty “longest so far” | `longest = ""` |
| 2. Open the file | `with open("trade.txt", "r") as f:` |
| 3. Go through it line by line | `for line in f:` |
| 4. Cut the line into words | `words = line.split()` |
| 5. Look at each word | `for word in words:` |
| 6. Longer than the longest so far? Keep it | `if len(word) > len(longest):` |
| 7. At the end, show the result | `print(longest)` |

PART 4 · THE TASK

# Version A: compare word by word

CODE

```
longest = ""

with open("trade.txt", "r") as f:
    for line in f:
        words = line.split()
        for word in words:
            if len(word) > len(longest):
                longest = word

print(longest)
print(len(longest))
```

OUTPUT

```
(hereinafter,
13
```

> The winner is “(hereinafter,” with a bracket and a comma. We fix that soon.

PART 4 · THE TASK

# What the loop does on line 1

| WORD | LEN(WORD) | LEN(LONGEST) | LONGER? | LONGEST IS NOW |
|---|---|---|---|---|
| The | 3 | 0 | yes | The |
| parties | 7 | 3 | yes | parties |
| to | 2 | 7 | no | parties |
| this | 4 | 7 | no | parties |
| Agreement | 9 | 7 | yes | Agreement |
| recognize | 9 | 9 | no, equal | Agreement |
| that | 4 | 9 | no | Agreement |
| trade | 5 | 9 | no | Agreement |

> Line 2 then brings “employment.” (11), and line 3 brings “(hereinafter,” (13).

PART 4 · THE TASK

# max() with key=len

CODE

```
words = ["trade", "Agreement", "WTO"]

print(max(words))              # alphabetical!
print(max(words, key=len))     # the longest
print(min(words, key=len))     # the shortest
```

OUTPUT

```
trade
Agreement
WTO
```

> key=len tells max() to compare the words by length, not by the alphabet.

PART 4 · THE TASK

# Version B: collect the words, then max()

CODE

```
all_words = []

with open("trade.txt", "r") as f:
    for line in f:
        all_words = all_words + line.split()

print(len(all_words))
print(max(all_words, key=len))
```

OUTPUT

```
25
(hereinafter,
```

> Same answer as version A. + joins two lists into one.

PART 4 · THE TASK

# Punctuation sticks to the words

CODE

```
all_words = []

with open("trade.txt", "r") as f:
    for line in f:
        for word in line.split():
            clean = word.strip('.,()"')
            all_words.append(clean)

print(max(all_words, key=len))
```

OUTPUT

```
hereinafter
```

strip('.,()"') removes those characters from both ends. append() adds one item to a list.

# 05

# Practice

*Small exercises. Answers follow at the end of this part*

PART 5 · PRACTICE

# Practice 1: strings

| | | |
|---|---|---|
| 1 | Store **"dispute settlement body"** in a variable. Print it in UPPER case and in Title Case. | **EXPECTED OUTPUT**<br>`DISPUTE SETTLEMENT BODY`<br>`Dispute Settlement Body` |
| 2 | Count how many times **"shall"** appears in **"Members shall notify and shall consult."** | **EXPECTED OUTPUT**<br>`2` |
| 3 | In **"Each Member shall notify the Committee."** replace "Member" with "Country". | **EXPECTED OUTPUT**<br>`Each Country shall notify the Committee.` |
| 4 | Split **"most favoured nation treatment"** into words. Print how many words there are. | **EXPECTED OUTPUT**<br>`4` |

PART 5 · PRACTICE

# Practice 2: loops

5

Loop over **["GATT", "GATS", "TRIPS", "DSU"]** and print each name with its length.

**EXPECTED OUTPUT**

```
GATT 4
GATS 4
TRIPS 5
DSU 3
```

6

From **["tariff", "quota", "ban", "duty"]** print only the words longer than 4 letters.

**EXPECTED OUTPUT**

```
tariff
quota
```

7

Use a counter variable and a loop to count the words in that same list.

**EXPECTED OUTPUT**

```
4
```

PART 5 · PRACTICE

# Practice 3: files

| | | |
|---|---|---|
| 8 | Open **trade.txt** and count how many lines it has. | **EXPECTED OUTPUT**<br>`3` |
| 9 | Print only the lines of trade.txt that contain the word **"Members"**. | **EXPECTED OUTPUT**<br>`Members shall treat like products (hereinafter, "products") equally.` |
| 10 | Find the **shortest** word in trade.txt. Hint: min() works like max(). | **EXPECTED OUTPUT**<br>`to` |
| 11 | Save a copy of trade.txt in capital letters as **trade_upper.txt**. Then open it and print it. | **EXPECTED OUTPUT**<br>`THE PARTIES TO THIS AGREEMENT ...` |

PART 5 · PRACTICE

# Solutions 1: strings

**EXERCISE 1**

```
name = "dispute settlement body"
print(name.upper())
print(name.title())
```

**EXERCISE 2**

```
text = "Members shall notify and shall consult."
print(text.count("shall"))
```

**EXERCISE 3**

```
text = "Each Member shall notify the Committee."
print(text.replace("Member", "Country"))
```

**EXERCISE 4**

```
text = "most favoured nation treatment"
words = text.split()
print(len(words))
```

PART 5 · PRACTICE

# Solutions 2: loops

### EXERCISE 5

```
names = ["GATT", "GATS", "TRIPS", "DSU"]
for name in names:
    print(name, len(name))
```

### EXERCISE 6

```
words = ["tariff", "quota", "ban", "duty"]
for word in words:
    if len(word) > 4:
        print(word)
```

### EXERCISE 7

```
words = ["tariff", "quota", "ban", "duty"]
count = 0
for word in words:
    count = count + 1
print(count)
```

### SHORTCUT

For exercise 7, len(words) gives the same answer in one step. The loop version is practice for the longest-word pattern.

PART 5 · PRACTICE

# Solutions 3: files

### EXERCISE 8

```
count = 0
with open("trade.txt", "r") as f:
    for line in f:
        count = count + 1
print(count)
```

### EXERCISE 9

```
with open("trade.txt", "r") as f:
    for line in f:
        if "Members" in line:
            print(line.strip())
```

### EXERCISE 10

```
all_words = []
with open("trade.txt", "r") as f:
    for line in f:
        all_words = all_words + line.split()
print(min(all_words, key=len))
```

### EXERCISE 11

```
with open("trade.txt", "r") as f:
    text = f.read()

with open("trade_upper.txt", "w") as f:
    f.write(text.upper())
```

# 06

# Regex

*Describe a pattern instead of exact text*

PART 6 · REGEX

# Why regular expressions?

## String methods

*find exact text*

- `"Article 3" in text`
- `text.count("2019")`
- `line.split()`

You must know the exact text. split() keeps the punctuation.

## Regular expressions

*find patterns*

- any 4-digit number (a year)
- any word, without punctuation
- any word with a capital letter

You describe what the text looks like, and Python finds every match.

PART 6 · REGEX

# Your first regex: find all numbers

CODE

```
import re   # load the regex module (once, at the top)

text = "The panel was set up in 2019 and reported in 2021."
print(re.findall(r"\d+", text))
```

OUTPUT

```
['2019', '2021']
```

`r"..."`

write every pattern as a raw string: r before the quotes

`\d`

one digit, 0 to 9

`+`

one or more of the thing before it

PART 6 · REGEX

# The building blocks

| SYMBOL | MEANS | PATTERN | MATCHES |
|---|---|---|---|
| `\d` | a digit | `\d\d` | 19, 20 |
| `\w` | a letter, digit or _ | `\w+` | trade, GATT, 1994 |
| `\s` | a space, tab or line break | `\s+` | one or more spaces |
| `.` | any character | `b.n` | ban, bin, b9n |
| `+` | one or more | `\d+` | 5, 12, 2024 |
| `{n}` | exactly n times | `\d{4}` | 1994, 2024 |
| `[ ]` | one of these characters | `[A-Z]` | any capital letter |

PART 6 · REGEX

# \w+ keeps the words, drops punctuation

CODE

```
import re
line = 'Members shall treat like products (hereinafter, "products") equally.'

print(line.split())
print(re.findall(r"\w+", line))
```

OUTPUT

```
['Members', 'shall', 'treat', 'like', 'products', '(hereinafter,', '"products")', 'equally.']
['Members', 'shall', 'treat', 'like', 'products', 'hereinafter', 'products', 'equally']
```

PART 6 · REGEX

# Four useful patterns

### Find years (4 digits)

```
text = "Signed in 1994, in force since 1995."
print(re.findall(r"\d{4}", text))
```

```
['1994', '1995']
```

### Find words with a capital

```
text = "The WTO and the European Union"
print(re.findall(r"[A-Z]\w+", text))
```

```
['The', 'WTO', 'European', 'Union']
```

### Find article numbers

```
text = "See Article 3 and Article 12."
print(re.findall(r"Article \d+", text))
```

```
['Article 3', 'Article 12']
```

### Find e-mail addresses

```
text = "Write to info@example.org today."
print(re.findall(r"\w+@\w+\.\w+", text))
```

```
['info@example.org']
```

PART 6 · REGEX

# Three functions: findall, search, sub

**SETUP**

```
import re
text = "Adopted in 1994.   Amended   in 2005."
```

### re.findall()

every match, as a list

```
print(re.findall(r"\d{4}", text))
```

```
['1994', '2005']
```

### re.search()

is there a match? use it in an if

```
if re.search(r"\d{4}", text):
    print("Found a year")
```

```
Found a year
```

### re.sub()

replace every match

```
print(re.sub(r"\s+", " ", text))
```

```
Adopted in 1994. Amended in 2005.
```

PART 6 · REGEX

# The longest word, cleaned with regex

CODE

```
import re

all_words = []
with open("trade.txt", "r") as f:
    for line in f:
        all_words = all_words + re.findall(r"\w+", line)

print(max(all_words, key=len))
```

OUTPUT

```
hereinafter
```

One change from version B: re.findall(r"\w+", line) instead of line.split().

PART 6 · REGEX

# Practice 4: regex

**R1** Find all numbers in **"Tariffs rose from 5 to 12 percent in 2024."**

**EXPECTED OUTPUT**

```
['5', '12', '2024']
```

**R2** Find all words that start with a capital letter in **"The European Union and Japan signed it."**

**EXPECTED OUTPUT**

```
['The', 'European', 'Union', 'Japan']
```

**R3** Replace every digit with # in **"Case DS543 and DS544"**.

**EXPECTED OUTPUT**

```
Case DS### and DS###
```

> Stuck? Look at the building blocks slide, and test your pattern at regex101.com.

PART 6 · REGEX

# Solutions 4: regex

**R1 · AFTER IMPORT RE**

```
text = "Tariffs rose from 5 to 12 percent in 2024."
print(re.findall(r"\d+", text))
```

**R2 · AFTER IMPORT RE**

```
text = "The European Union and Japan signed it."
print(re.findall(r"[A-Z]\w+", text))
```

**R3 · AFTER IMPORT RE**

```
text = "Case DS543 and DS544"
print(re.sub(r"\d", "#", text))
```

# What you can do now

- Change and clean strings: upper(), lower(), strip(), replace(), split()
- Open, read and write text files with with open(...)
- Loop through a file line by line
- Remember a value across a loop: create, update, use
- Find the longest word in a file, in two different ways
- Recognise when a regex pattern makes the job easier

**Keep learning**

Python string methods

Python regex HOWTO

regex101: test patterns online

*Small steps, typed by you, every day: that is how code becomes familiar.*